



System Design

Given a scenario, compare, contrast, and recommend the components and tools of a successful deployment strategy.

Develop your career with Salesforce Training and Certification Preparation

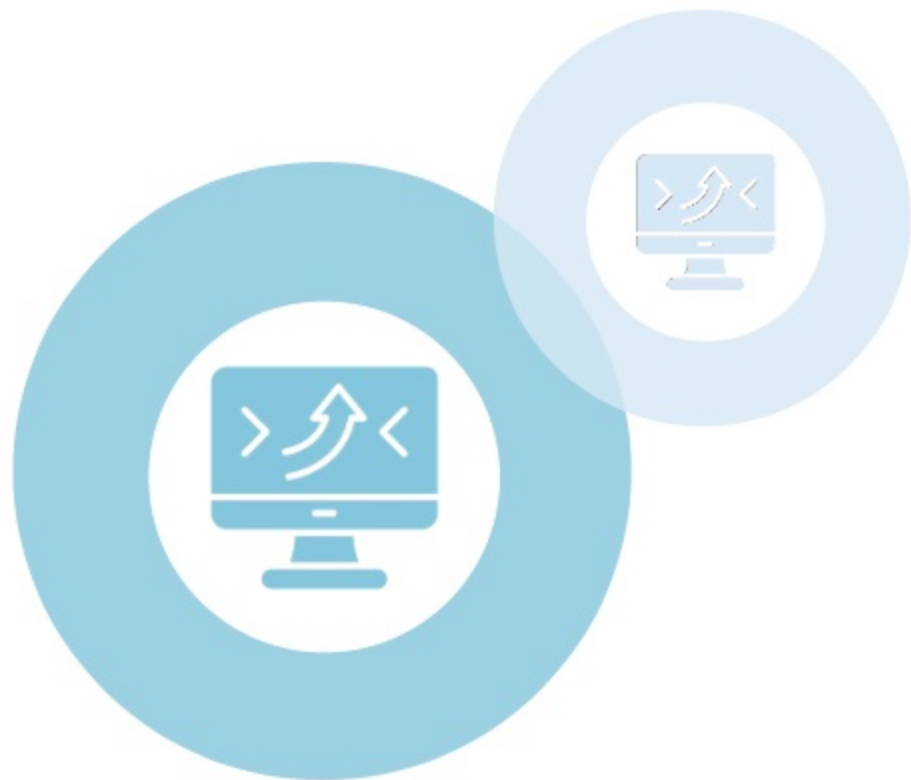
Table of Contents

-  [Deployment Components](#)
-  [Deployment Tools](#)
-  [Business Scenario](#)
-  [Requirements & Solutions](#)



After studying this topic, you should be able to:

-  Describe the different deployment components for a successful deployment strategy.
-  Describe the different deployment tools for a successful deployment strategy.
-  Recommend the deployment components and tools the given business requirements.



Introduction

For a successful deployment strategy, it is important to consider various components and tools. **Deployment components** include metadata components, data components, governance, change management, test plans and scripts, and rollback strategy.

On the other hand, there are various **native and third-party deployment tools**. They include Salesforce DX, Salesforce CLI, Metadata API, Ant Migration Tool, Change Sets, packages, automated testing tools, and CI/CD tools.



Deployment Components

Metadata & Data

Metadata components are building blocks of customizations that are moved between Salesforce environments. Data components are records of Salesforce objects that are migrated between orgs.



Governance & Change Management

A governance framework is a set of policies and procedures that guide how changes are managed and deployed. It includes tracking, reviewing, and approving changes.

Test & Rollback Strategies

Test plans and scripts are used to ensure that a deployment does not negatively affect functionality. A rollback strategy can be used to revert changes if a deployment causes issues in production.

Deployment Tools



Deployment Components

[Table of Contents](#)



Deployment Components

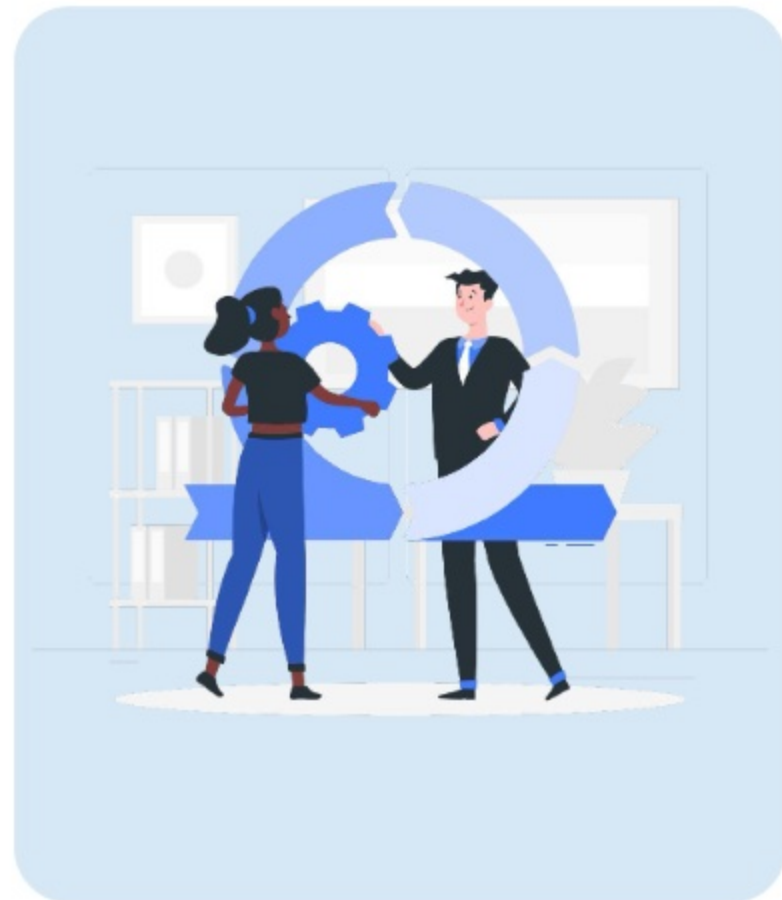
The components of a successful Salesforce deployment strategy focus on aspects such as **metadata**, **data**, **governance**, **testing**, and **rollback**.

❖ METADATA COMPONENTS

Metadata components are the **building blocks of Salesforce customization**, such as custom objects, custom fields, Apex classes, Lightning web components, Visualforce pages, etc.

❖ DATA COMPONENTS

Data components represent **records of Salesforce objects** that need to be migrated between environments.



Deployment Components

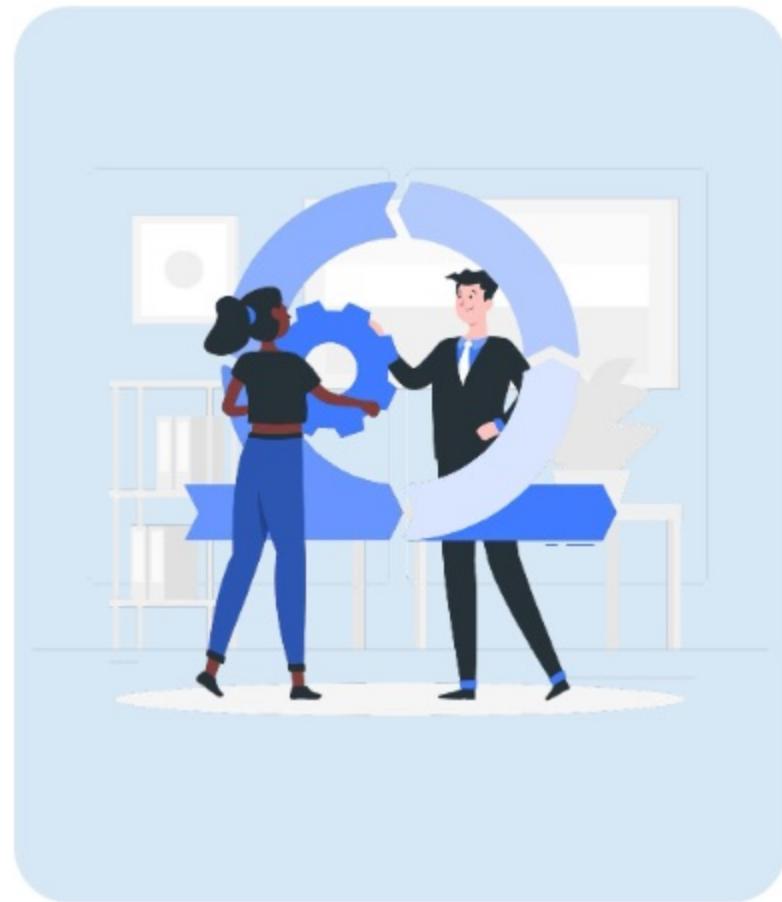
The components of a successful Salesforce deployment strategy focus on aspects such as **metadata**, **data**, **governance**, **testing**, and **rollback**.

◆ GOVERNANCE

A governance framework is a set of **policies and procedures** that guide how **changes are managed and deployed** across different environments.

◆ CHANGE MANAGEMENT PROCESS

A change management process focuses on establishing procedures for **tracking, approving, and deploying changes**, ensuring that all stakeholders are aligned and informed.



Deployment Components

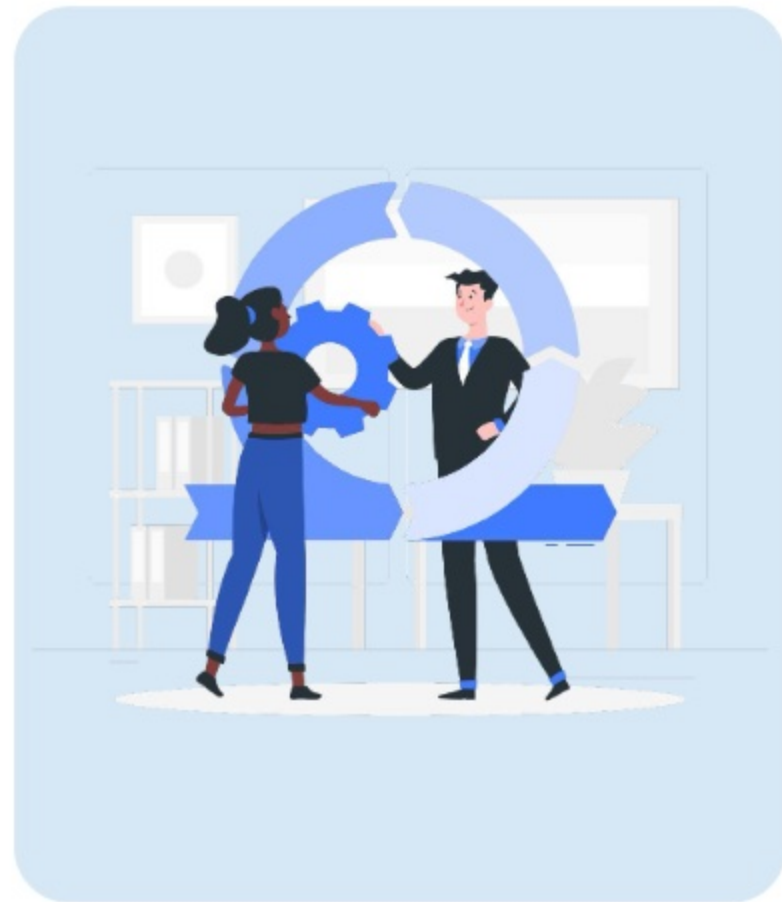
The components of a successful Salesforce deployment strategy focus on aspects such as **metadata**, **data**, **governance**, **testing**, and **rollback**.

❖ TEST PLANS & SCRIPTS

Test plans and scripts are **detailed documents** used to ensure that a deployment **does not negatively impact** the existing functionality.

❖ ROLLBACK STRATEGY

A **contingency plan** is necessary to **revert changes** if a deployment causes **unexpected issues** in the production environment.



Deployment Tools

Table of Contents



Deployment Tools

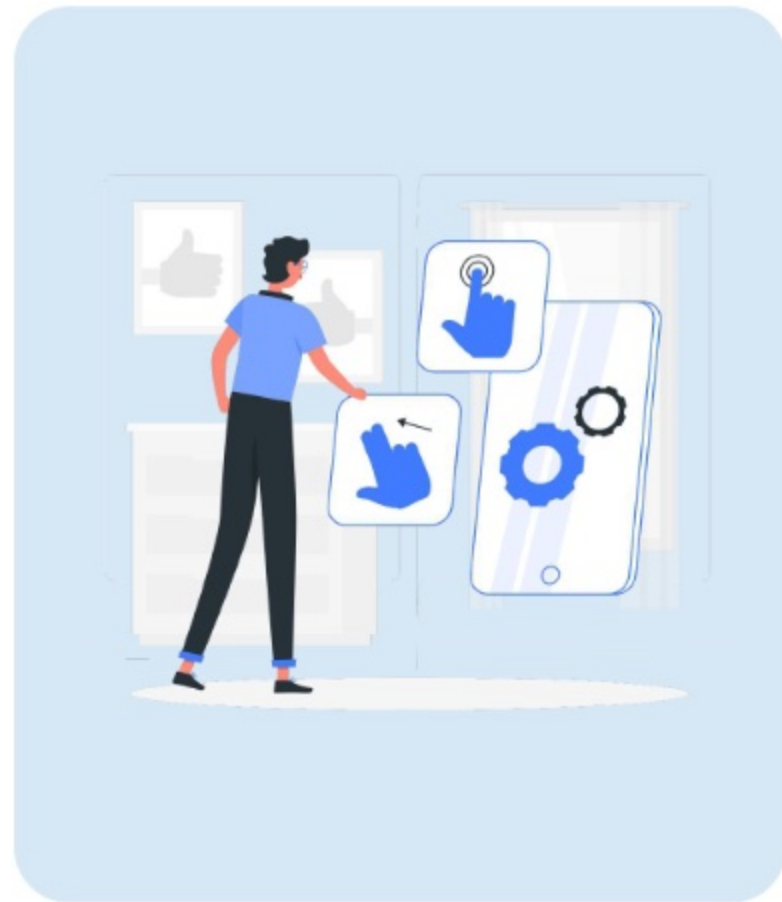
Various tools are available for **deploying changes** in Salesforce. These include tools offered by **Salesforce** as well as **third-party tools**.

◆ SALESFORCE DX

Salesforce DX is a **suite of tools** used to **build, test, and deploy** applications quickly and efficiently. It facilitates **source-driven development**, using **scratch orgs** for **development and testing**, and **package development** for easy distribution and installation.

◆ SALESFORCE CLI

Salesforce CLI is a **command-line interface** that can be used for deployment using **Salesforce Extensions for VS Code or Code Builder**.



Deployment Tools

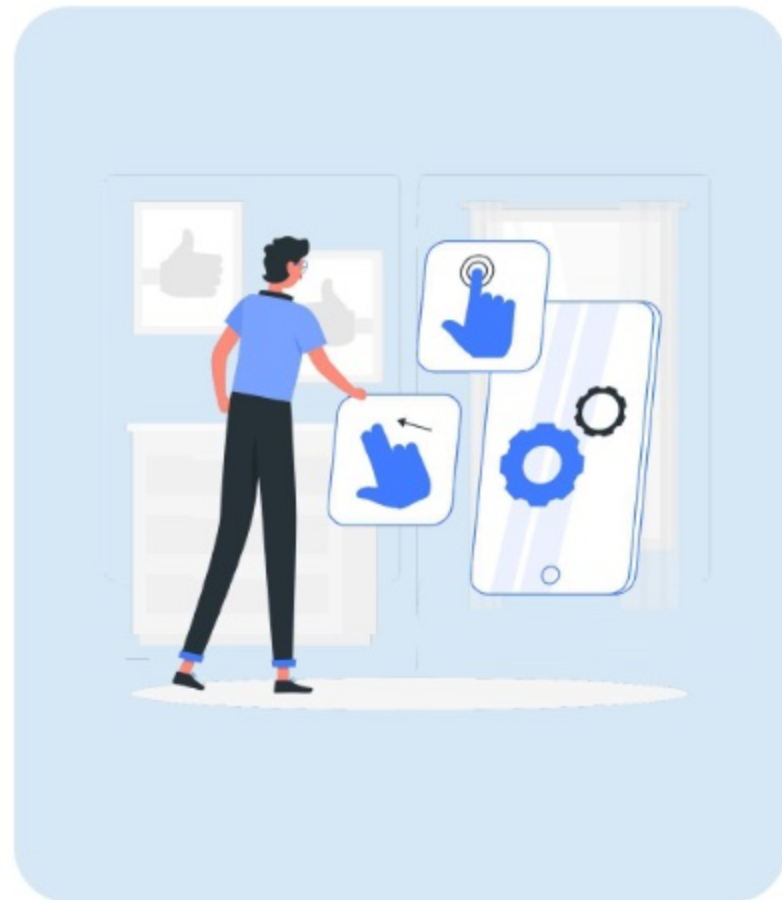
Various tools are available for **deploying changes** in Salesforce. These include tools offered by **Salesforce** as well as **third-party tools**.

❖ METADATA API

Metadata API can be used to **retrieve, deploy, create, update or delete** customization information.

❖ ANT MIGRATION TOOL

The Ant Migration Tool is a **Java/Ant-based command-line utility** used to **move metadata** between a **local directory** and a **Salesforce org**.



Deployment Tools

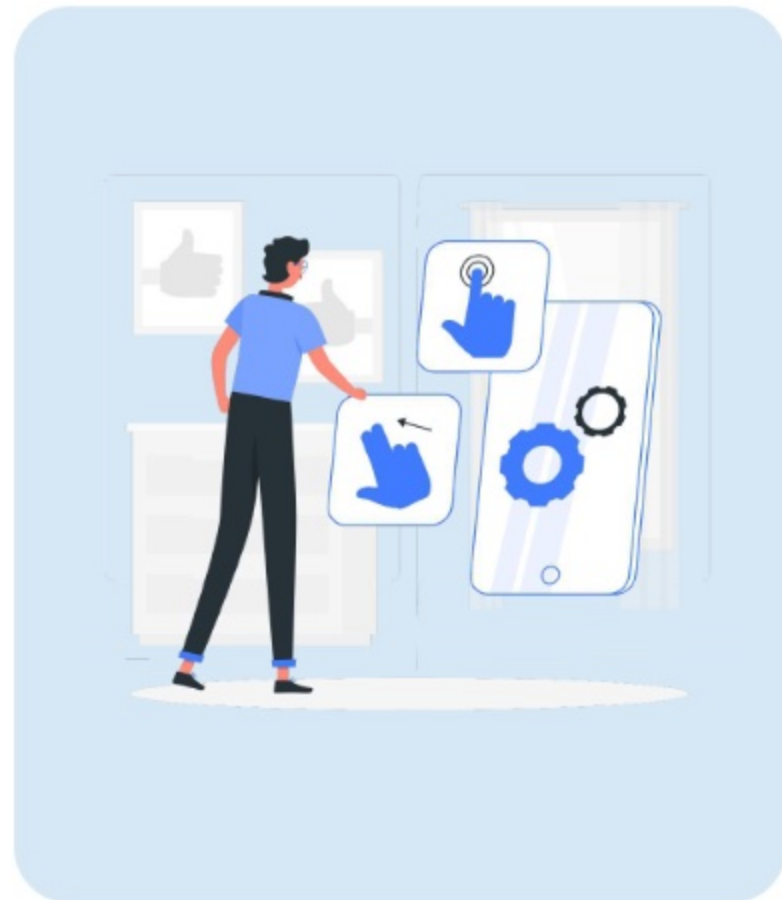
Various tools are available for **deploying changes** in Salesforce. These include tools offered by **Salesforce** as well as **third-party tools**.

❖ CHANGE SETS

Change sets are **native Salesforce tools** used for **deploying metadata between connected Salesforce orgs**. They are suitable for smaller, less complex deployments.

❖ PACKAGES

Packages can be used to deploy code and metadata changes from one org to another. **Unmanaged packages** are open-source solutions while **managed packages** are customizable and upgradable.



Deployment Tools

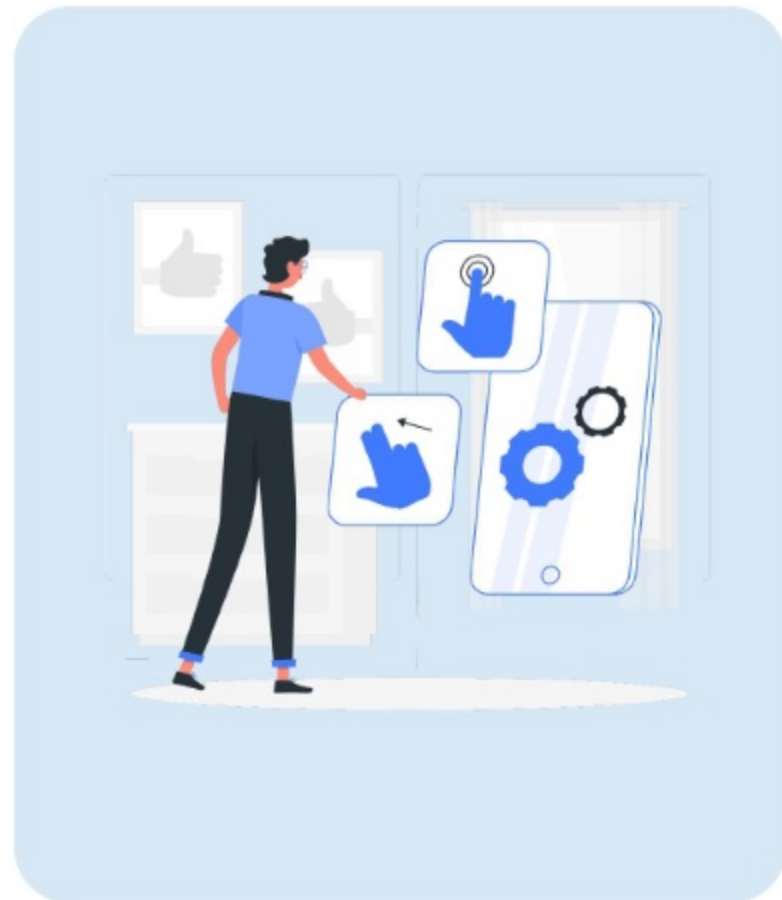
Various tools are available for **deploying changes** in Salesforce. These include tools offered by **Salesforce** as well as **third-party tools**.

❖ AUTOMATED TESTING TOOLS

Third-party automated testing tools, such as **Selenium**, can be used to automate tests in sandbox environments.

❖ CI/CD TOOLS

Third-party **CI/CD (Continuous Integration/Continuous Deployment)** tools like **Copado** and **Flosum** are often used to automate the build and deployment processes.



Salesforce DX

Salesforce Developer Experience (DX) supports **building and deploying applications** on the Salesforce platform in a more efficient, flexible, and controller manner.



Salesforce DX enabled **source-driven development**. Source code and metadata exist in a version control system.



It facilitates **continuous integration and delivery** using tools such as CircleCI, Jenkins, and Travis CI.



Salesforce CLI automates various tasks, from **org management to data import/export**, streamlining the deployment process.



Metadata API can be used to **retrieve and deploy changes** using the development workflow called the **org development model**.



Unlocked packages can be used to organize existing metadata, package an app, extend an app, or package new metadata.

Salesforce CLI

Salesforce CLI is a **command-line tool** that is part of Salesforce DX.



Salesforce CLI streamlines various tasks, such as creating and managing **Salesforce environments**, pushing and pulling **source code**, executing **tests**, and deploying **metadata**.



The CLI provides **commands** for working with **data** and **metadata**, which allow for the automation of **deployment processes**.



Salesforce CLI commands make it easier to integrate with **Continuous Integration (CI) systems** for **automated testing and deployment**.

Metadata API

Metadata API can be used to **move metadata between orgs** during the development lifecycle and **deploying large metadata changes** from development.



Metadata can be moved from **production to the local file system** to make development changes.



Metadata changes can be **moved to and from a scratch org**, which can be used along with the local file system for development and testing.



Metadata can be **moved to sandboxes** for **integrating changes, testing, and collaboration.**



In the final step of the development cycle, metadata can be **moved from a source control system such as Git to production.**



Metadata can be moved for **large changes**, such as **splitting or merging production orgs.**

Ant Migration Tool

The Ant Migration Tool is a **Java/Ant-based command-line utility** used for **moving metadata** between **a local directory and a Salesforce org**.



The tool can be used for development projects that require **populating a test environment** with a lot of Setup changes.



It can be used for **scripted retrieval and deployment** of components, making **multi-stage release processes** much more efficient.



It can be used for **repetitive deployments** using the **same parameters**. The same process for deploying changes can be used.



It can be used for **deployments in a scripting environment**, such as for migrating changes from staging to production.



Batch deployments can be **scheduled**. For example, a deployment can be scheduled for midnight or changes can be pulled every day.

Change Sets

Change Sets can be used to **move customizations between Salesforce orgs** that are **affiliated with a production org**.



Once a **deployment connection** is created, an **outbound change set** can be **created** and **uploaded** to send changes to another org.



When selecting components for an outbound change set, **dependent components** and **profile settings** can be added.



The receiving org sees the outbound change set as an **inbound change set**, which must be **deployed** for the changes to take effect.



A change set can be **validated** before deploying changes. Validation allows viewing **success or failure messages** received with an actual deployment.



The deployment of change sets can be tracked on the **Deployment Status** page. The **change set detail page** shows completed deployments.

Change Sets

Change Sets can be used to **move customizations between Salesforce orgs** that are **affiliated with a production org**.



Inbound changes from an org can be authorized by selecting **Allow Inbound Changes** on the Deployment Settings page in Setup.



The use of change sets **cannot be automated** based on metadata differences in an org.



The **originating org is locked** while **validating** an outbound change set. The **target org is locked** while **deploying** an inbound change set.



When an org is locked, it is possible to **read and write data**, but any Setup changes that would modify the metadata are not possible.



Only **one version of a flow** can be included in a change set. If there is no active version, **the latest inactive version** is used.

Packages

Unlocked packages can be utilized to **organize and deploy metadata** for internal business applications across different environments.



Each unlocked package has a distinct **life cycle**. A new **package version** is created after adding, removing or changing package metadata. A package version is an **immutable artifact** that contains specific metadata and features.



A package version can be installed in various Salesforce environments, including **sandboxes** and **scratch orgs**, facilitating testing and staging before deployment to production.



Unlocked packages can be integrated with a **version control system** like Git. Version control allows **deploying updates** and **rolling back changes** with greater precision.

Automated Testing Tools

Many **third-party automation tools**, such as Selenium, are available for Salesforce testing. They **standardize testing processes** and **reduce the risk of human error**.



Automated testing tools ensure that **deployments are successful** by **identifying potential issues** before they affect users. They automate the testing process, allowing for quicker deployment.



The **common types of tests** that can be automated include unit tests, functional tests, integration tests, regression tests, load tests, and UI tests.



Advantages of automated testing include increased **test coverage**, early **bug detection**, **time and cost savings**, **cleaner existing code**, **reliable deployments**, and **reduced human errors**.

CI/CD Tools

Third-party CI/CD tools **streamline development workflows** and are crucial for **continuous integration, continuous deployment**, and **continuous delivery**.



Continuous Integration (CI)

focuses on automatically moving work items through a release pipeline.



Continuous Deployment (CD)

focuses on automatically deploying changes to a downstream environment.



A **change** is deployed as soon as it has been **approved and merged** into **version control**, ensuring the latest changes.



Continuous Delivery is the ultimate goal of CI/CD. It focuses on releasing added value and new functionalities as soon as possible.

CI/CD Tools

Options for continuous integration and continuous deployment include **standard automation tools** and **Salesforce-specific DevOps platforms**.



CI/CD **automates** repetitive, error-prone manual processes, increases **development velocity**, and significantly **saves developer time**.



A successful CI/CD process is usually rooted in **version control** as the **source of truth** for new development work.



Automation is used to **validate** deployment packages, **carry out tests**, and **promote work items** along the pipeline.



Standard automation tools include **CircleCI**, **Jenkins**, and **Travis CI**. Major Git hosting platforms also offer CI/CD capabilities.



Salesforce-specific tools like **Copado** and **Flosum** offer CI/CD as part of their DevOps solutions.

Business Scenario

Table of Contents



Business Scenario

Read the business scenario below and determine the most appropriate solutions for the company's requirements.



Cosmic Tech Solutions, a leading software development company that specializes in cloud-based applications, is undergoing a strategic shift to enhance its deployment processes across its Salesforce environments. The company currently utilizes a diverse set of Salesforce clouds, including Sales Cloud for CRM, Service Cloud for customer service management, and custom applications built on the Lightning platform to cater to specific business needs.

With a growing global presence and an increasing number of complex projects requiring frequent releases, the company faces the challenge of streamlining its deployment strategy to ensure efficient, error-free releases that align with business objectives and technical requirements.

The company's Salesforce architect is required to evaluate the existing deployment processes and recommend the components and tools for various business requirements to establish a successful deployment strategy.

Requirements & Solutions

Table of Contents 

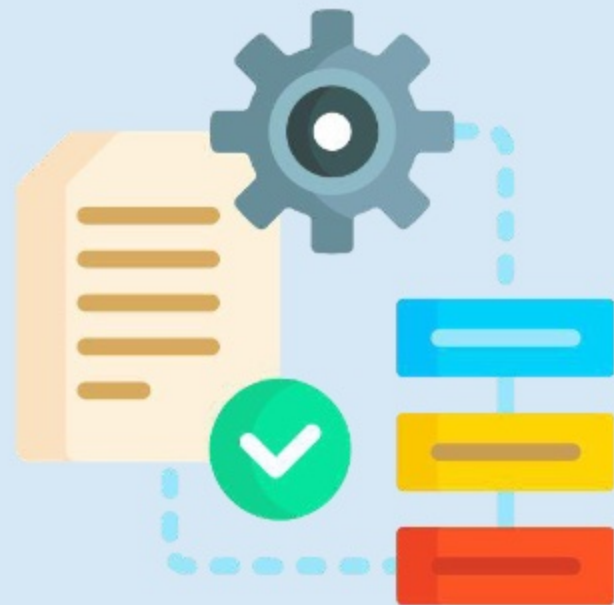
Requirement 1



SCENARIO

The company would like to streamline and automate the deployment of customizations to various sandbox environments during the build phase. Once user acceptance testing is complete in the UAT environment, the deployment to the production org should also be automated.

Also, metadata changes need to be tracked in Salesforce throughout the development lifecycle across various environments. Furthermore, the company is looking for ways to reduce the deployment time and wants a rollback strategy for new features deployed to production.



Solution 1 (Part 1)



SOLUTION

In order to streamline and automate the deployment of customizations to various sandbox environments during the build phase, a **CI/CD tool** such as Jenkins or a DevOps platform such as Copado should be utilized. A **release pipeline** can be built to deploy to sandboxes and production.

The CI/CD tool can automatically move work items through the release pipeline, combining and testing changes. It can also automatically deploy a change to a downstream environment once it has been approved and merged into version control. Metadata changes can be tracked in Salesforce using a **third-party tool** available in the **AppExchange marketplace**.



Solution 1 (Part 2)



SOLUTION

Before the deployment of a new feature to production, a **sandbox** can be created from production for a **backup of the existing metadata**. After the deployment of the new feature, if a rollback is required, **destructiveChanges.xml** can be used to delete the new components, and metadata from the sandbox can be deployed from the sandbox to production.

To reduce the deployment time, **recent deployment validations** and the **Quick Deploy** feature can be utilized. Also, specific tests can be run for deployments by using the **RunSpecifiedTests** test level.

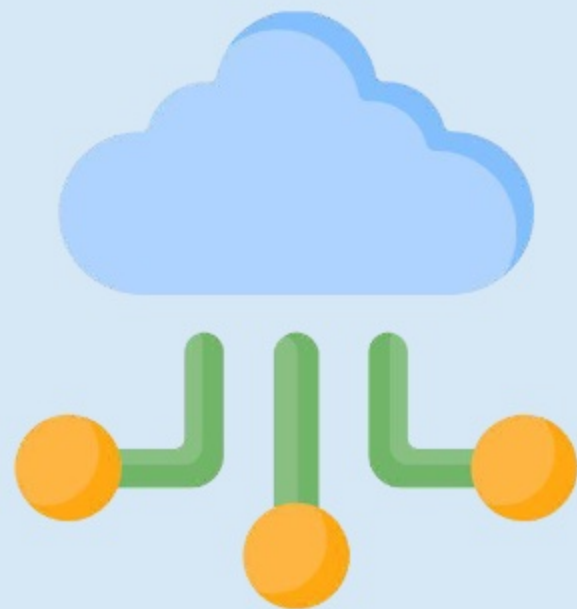


Requirement 2



SCENARIO

The company has set up two new regional production instances for its operations in APAC and EMEA. A development team in the United States has created a custom application for managing different types of software solutions that needs to be deployed to the new production instances. The regional development teams should be able to maintain and modify the application metadata. Also, the company wants to establish a deployment plan for releasing simple configuration changes and enhancements, such as a new flow. The architect needs to recommend a suitable tool while outlining its benefits and any valid considerations related to automation and deployment.



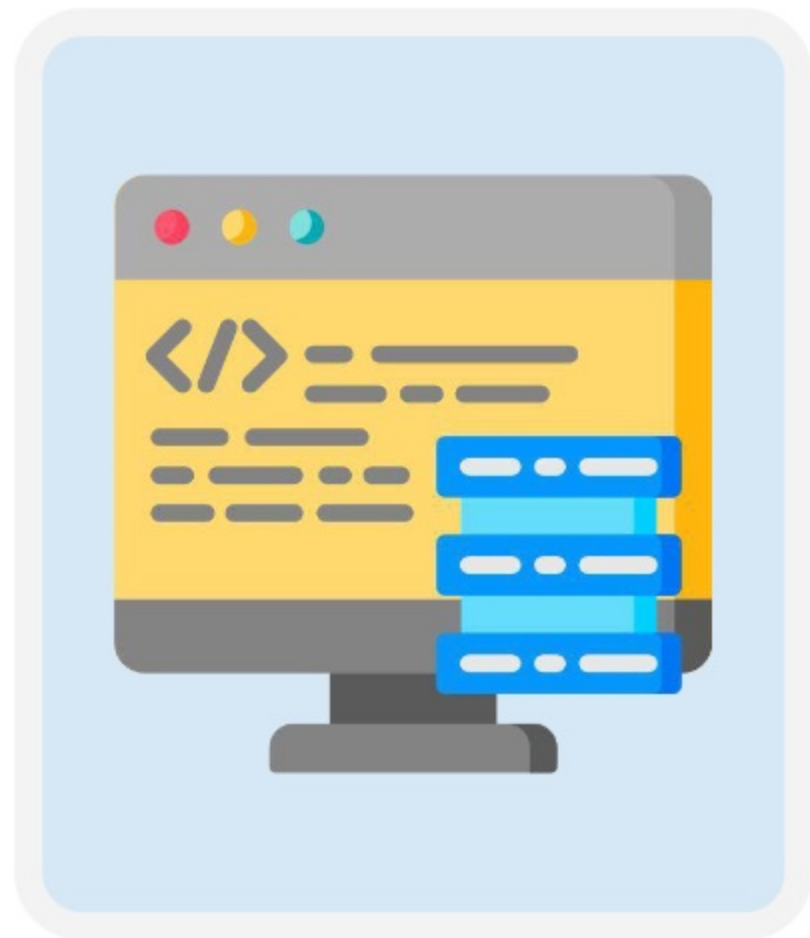
Solution 2 (Part 1)



SOLUTION

Deployment tools such as the [Ant Migration Tool](#) or [Salesforce Extensions for VS Code](#) can be utilized to deploy the custom application metadata to the new production instances.

For simple configuration changes, such as a new flow, [change sets](#) can be used to migrate the metadata between different environments. They offer an easy and declarative way of deploying components, including dependent components, [between orgs that are affiliated with a production org](#). To use change sets, a deployment connection must be established. An org that needs to receive customizations through a change set must [allow inbound changes](#) from the source org.

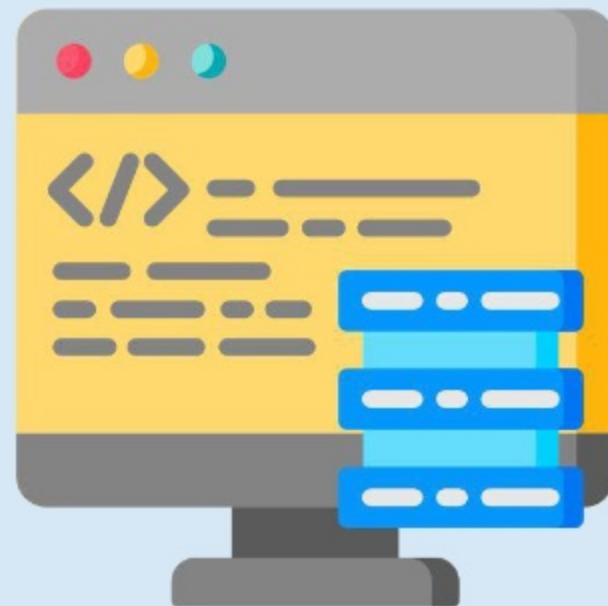


Solution 2 (Part 2)



SOLUTION

There are other important considerations for using change sets, especially for migrating a flow. The use of change sets **cannot be automated** based on metadata differences in an org. Only **one version of a flow** can be included in a change set, and if there is no active version, **the latest inactive version** is used. Although changes sent using a change set can be validated, the **originating org is locked** while **validating** an outbound change set. The **target org is locked** while **deploying** an inbound change set. When an org is locked, it is possible to **read and write data**, but any Setup changes that would modify the metadata are not possible.



Learn More

-  [How Salesforce Developer Experience Changes the Way You Work](#)
-  [Use Cases for Metadata API](#)
-  [Ant Migration Tool](#)
-  [Unlocked Packages](#)
-  [Choose Your Tools for Developing and Deploying Changes](#)

